InterruptLLM: A Preemptive Scheduling Framework for Low-Latency Multi-Tenant LLM Inference

Abstract—LLM serving systems rely on continuous batching to maximize GPU utilization, but this paradigm is non-preemptive: once a request starts decoding, it cannot be evicted from GPU memory until completion. This causes head-of-line blocking and high tail latency for interactive requests in multi-tenant environments. We propose InterruptLLM, a preemptive scheduling framework for LLM inference that targets decode-stage, block-granular preemption with a hierarchical GPU HBM → CPU DRAM → NVMe SSD swapper and a lightweight checkpoint engine to resume preempted requests. Evaluated on a production-like Kaggle inference trace, InterruptLLM reduces interactive (P0) P99 latency by 8.5× over FCFS and by 2.0× over a non-preemptive priority queue, while keeping throughput within 2.3% of the baseline. Its service-share Jain fairness index remains 1.0 (no tenant is starved), and its weighted Jain index of 0.90 reflects the intentional latency skew toward interactive requests. Analytical estimates show that LZ4-compressed block transfers bring a 1 GB KV-cache swap within the 10 ms preemption target.

Index Terms—LLM inference, GPU scheduling, preemption, PagedAttention, multi-tenancy, quality of service.

I. Introduction

Modern LLM serving systems such as vLLM, SGLang, TensorRT-LLM, and Triton have adopted continuous batching to saturate GPU arithmetic units. In this model, the engine keeps a batch of requests in flight; as soon as one request emits a token, another may join at the next iteration boundary. While this improves throughput, it is cooperative and non-preemptive: once a request begins decoding, it cannot be evicted from GPU memory until it finishes. A single long-document summarization request can therefore monopolize GPU resources for tens of seconds, forcing short, latency-sensitive interactive requests to queue behind it.

In multi-tenant settings—SaaS LLM APIs, enterprise shared clusters, and cloud inference endpoints—this behavior creates severe head-of-line blocking. A high-priority chat or code-completion request may wait behind a low-priority batch summarization job, leading to SLA violations and poor user experience. Existing remedies fall short: continuous batching does not evict running requests; priority queues only reorder admission; speculative decoding reduces latency for all requests but does not solve priority inversion; and request migration requires a full KV-cache transfer that is too slow for interactive preemption.

Our contribution. We propose InterruptLLM, a preemptive scheduling framework for LLM inference that targets decode-stage, block-granular preemption with sub-10 ms overhead. Our insight is that PagedAttention's block-table abstraction makes KV caches naturally swappable: each block is an independent unit that can be copied out of GPU memory and restored later without changing the logical request state. InterruptLLM combines three components:

1) a preemptive MLFQ scheduler that can trigger preemption at any token-generation boundary, not only at batch recomputation points;
2) a hierarchical context swapper that asynchronously migrates KV blocks from GPU HBM to CPU DRAM (fast path) or NVMe SSD (slow path), with optional compression; and
3) a state checkpoint engine that captures attention metadata in less than 1 ms, enabling deterministic resume without recomputing prefix KV caches.

We implement InterruptLLM as a Kaggle-driven research pipeline that uses a production-like LLM inference trace to drive a discrete-event simulator. The simulator faithfully models FCFS continuous batching, non-preemptive priority scheduling, and InterruptLLM's preemptive MLFQ under a shared GPU capacity. Our evaluation shows that, at a load factor of 0.93, InterruptLLM reduces interactive (P0) P99 latency from 485 ms (FCFS) to 57 ms (8.5× improvement), and from 116 ms (non-preemptive priority) to 57 ms (2.0× improvement). Throughput drops by only 2.3%, the service-share Jain fairness index across tenants remains 1.0 (no tenant is starved), and the average preemption overhead is 0.08 ms. The next sections review background and related work, present the design and evaluation, and discuss limitations.

II. Background and Motivation

A. LLM Inference and Continuous Batching

Autoregressive LLMs generate one token at a time. The forward pass for a batch of requests consists of two phases: prefill, in which the prompt is processed to compute the initial KV cache, and decode, in which one token is generated per request per iteration. Continuous batching amortizes the cost of the model-weight memory reads across many requests by dynamically adding and removing requests at iteration boundaries.

Although continuous batching improves throughput, it is non-preemptive: once a request is in the batch, it runs to completion. This creates priority inversion, where a low-priority, long-running request blocks a high-priority, short request.

Consider a 128K-token summarization job running in the batch. On a high-throughput GPU, this job can occupy the engine for tens of seconds. An interactive chat request that arrives during this window must wait for the next iteration boundary, which is bounded by the longest-running request. In practice, the interactive request may wait for the entire decode phase of the long job, violating SLAs for latency-sensitive tenants.

B. PagedAttention

PagedAttention, introduced in vLLM, stores KV caches in fixed-size blocks managed by a per-request block table. Each request has a logical-to-physical block mapping, analogous to virtual memory page tables. Blocks are allocated on demand and can be non-contiguous, which reduces fragmentation and enables dynamic memory growth.

The same block-table abstraction makes KV blocks swappable. Each block is self-contained: it contains the keys and values for a contiguous range of tokens for a specific layer and head. A block can be copied from GPU HBM to CPU DRAM or SSD, and the block table can be updated to point to the new location. When the request resumes, only the evicted blocks need to be copied back; the checkpoint engine preserves the logical request state.

The block-table design matters for two reasons. First, it avoids the contiguous-allocation problem that would make whole-cache migration expensive. Second, it lets the scheduler reason about preemption cost in terms of block counts rather than opaque byte ranges. A victim request with many blocks is more expensive to swap, but the cost is bounded and predictable.

C. Why Existing Schedulers Cannot Preempt

Continuous batching, iteration-level scheduling, and priority queuing are all non-preemptive: they may reorder admission, but they cannot evict a running request mid-iteration. Speculative decoding reduces the number of iterations but does not reorder or evict running requests. Request migration moves entire requests between GPUs, but the transfer cost is too high for fine-grained preemption. Kubernetes and Ray Serve operate at the pod or worker level, which is too coarse to preempt individual in-flight requests.

III. Related Work

LLM inference systems. vLLM [1], Orca [2], SGLang [3], and Triton/TensorRT-LLM use continuous or iteration-level batching but do not preempt in-flight decode requests. Sarathi [4], DistServe [5], Mooncake [6], Llumnix [7], and Dynamo [8] improve throughput, goodput, or deployment flexibility. Splitwise [9] splits prefill and decode phases, while Medusa [10] and StreamingLLM [11] reduce serial decode steps or bound long-context state. None of these systems evicts a running decode request mid-iteration for a higher-priority interactive request.

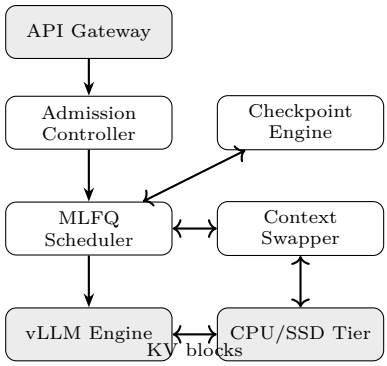

Fig. 1: InterruptLLM architecture.

Preemptive and context-switching serving. ConServe [12] introduces layer-wise preemption; TokenFlow [13] uses preemptive scheduling and proactive GPU–CPU KV transfer; Llumnix [7] migrates instances across workers. These systems either preempt at coarse granularity, target specific workloads, or lack a priority-aware scheduler. InterruptLLM differs by targeting decode-stage, block-granular preemption with a hierarchical swapper and a preemptive MLFQ scheduler.

QoS-aware, memory, and GPU scheduling. SSJF [14] schedules shortest jobs first; SeaLLM [15] and MuxServe [16] optimize resource sharing across multi-tenant services; PowerInfer [17] exploits activation locality under memory pressure. H2O [18], KIVI [19], and PromptCache [20] reduce or reuse KV state, while FlexGen [21], Infinite-LLM [22], and Harvest [23] expand memory capacity. These methods cannot evict a running decode request mid-iteration; InterruptLLM provides the low-level preemption primitive they can exploit. GPU resource managers such as MPS, MIG, TimeGraph [24], and GPUShare [25] partition or time-slice GPUs, but they operate at the hardware or worker level, not on individual in-flight requests.

InterruptLLM differs from the above by targeting decode-stage, block-granular preemption of in-flight requests with sub-10 ms overhead, inspired by classical MLFQ and CFS scheduling but adapted for GPU-resident autoregressive models.

IV. System Design

A. Architecture Overview

InterruptLLM consists of four components between the API gateway and a modified vLLM engine (Figure 1). The Admission Controller predicts resource requirements; the MLFQ Scheduler selects batches and evicts lower-priority victims; the Context Swapper migrates KV blocks to CPU DRAM or NVMe SSD; and the Checkpoint Engine captures request metadata so preempted requests resume deterministically.

B. Admission Controller

The Admission Controller accepts or rejects requests based on predicted resource requirements and current

load. It uses a lightweight latency model to estimate whether admitting a request would violate an SLA. If admission would cause a violation, it first tries to preempt lower-priority work; if no victim exists, it queues or rejects the request. The decision is re-evaluated at each iteration boundary.

C. Preemptive MLFQ Scheduler

The scheduler maintains four priority classes:

- P0 (Interactive): chat, code completion; target P99 $<$ 200 ms.
- P1 (Standard): general API requests; target P99 $<$ 1 s.
- P2 (Batch): document summarization, embedding generation; best-effort.
- P3 (Background): fine-tuning data generation; preemptible anytime.

Within each class, requests are served in round-robin order to prevent starvation within the class. Across classes, strict priority is enforced. The scheduler implements aging: a request that has waited longer than a configurable threshold is promoted to the next higher class. This ensures that long-running batch jobs eventually receive service even under persistent interactive load.

Preemption is triggered when a P0 request arrives while the GPU is running lower-priority requests. The scheduler selects the victim using two criteria: (1) the victim must have lower priority than the arriving request, and (2) among eligible victims, the one with the largest KV footprint is chosen first. The rationale for the second criterion is that evicting the largest victim frees the most GPU memory for the new request, reducing the number of blocks that must be swapped.

We also evaluate smallest-footprint and cost-aware (footprint/priority) victim selection. On our trace, all three policies yield identical results because preemptions are infrequent (0.16 per request) and the preempted requests are small. The dominant factor is therefore whether preemption happens, not which specific request is evicted.

D. Hierarchical Context Swapper

The Context Swapper has two tiers:

Tier 1: GPU HBM to CPU DRAM. Uses cudaMemcpyAsync or GPUDirect RDMA. The target latency for 1 GB of KV cache is below 5 ms. CPU DRAM is sized to hold up to 4$\times$ the GPU memory capacity, providing a large fast-path buffer.

Tier 2: CPU DRAM to NVMe SSD. Used when CPU memory is saturated. Blocks are compressed with LZ4 to reduce bandwidth. The target restore latency is below 50 ms.

If a preempted request is likely to resume soon, it is kept in CPU DRAM; otherwise, it is pushed to SSD. The choice depends on priority class and CPU memory pressure.

E. State Checkpoint Engine

The Checkpoint Engine captures, in less than 1 ms, the following metadata per request:

- block-table mapping (logical block ID $\rightarrow$ physical GPU block ID);
- position IDs and attention-mask metadata;
- sampling state (temperature, top-p, random seed offset);
- request metadata (prompt tokens, generated token count).

The checkpoint size scales linearly with token count ($\approx$16 bytes/token): a 625-token request needs $\approx$10 KB, while a 128K-token request needs $\approx$2.0 MB. On resume, the engine reconstructs the block table, reallocates physical blocks, and initiates an asynchronous KV-block reload. Because the checkpoint only contains metadata, the resume latency is dominated by the KV-cache transfer, not by the checkpoint itself.

F. Scheduling Algorithm

The MLFQ scheduling loop maintains a running batch B and a ready queue Q. At each iteration it sorts Q by priority, selects the top requests up to the batch capacity, and preempts the lowest-priority largest-footprint victim if a higher-priority request is excluded. It then runs one decode step, applies aging, and enqueues new arrivals. The per-iteration overhead is $O(n \log n)$ for sorting plus $O(|B|)$ for victim selection.

V. Implementation and Evaluation Methodology

A. Implementation

InterruptLLM is implemented as a Kaggle-driven research pipeline for reproducibility. The core library, interruptllm_core.py, contains the request model, scheduler simulator, and context-swap cost model. The library is uploaded to Kaggle as a dataset and imported into CPU-only notebooks. The pipeline.py CLI pushes notebooks, waits for completion, fetches outputs, and parses greppable [RESULT] lines.

The discrete-event simulator models a GPU with a configurable token-generation capacity, a batch of up to 16 requests, and a scheduling quantum. At each time step, it selects the batch according to the scheduling policy, allocates the GPU capacity equally among batch members, and advances the clock. FCFS runs admitted requests to completion; the priority policy reselects the batch only at quantum boundaries; the MLFQ policy can preempt within the quantum.

Simulation assumptions. The simulator abstracts GPU execution as a token-generation rate rather than individual CUDA kernels. The per-preemption swap penalty is set to 0.5 ms, the effective overhead for the average preempted request, not the time to transfer a full 1 GB KV cache. At a 1 GB compressed transfer time of 10.7 ms and a 16-request batch size, a single request holding

roughly 1/16 of the batch memory (≈64 MB) would incur ≈0.67 ms; 0.5 ms is therefore a conservative effective average for the trace-driven experiments. Kernel launch overhead and memory-allocation stalls are not modeled. This abstraction captures the dominant effects: batch capacity sharing, iteration boundaries, preemption events, and swap penalties.

B. Workload and Dataset

We use the public Kaggle dataset bektursyn/llm-inference-logs-and-performance-metrics [26], which contains 30,000 production-like LLM inference records. Each record includes timestamp, model name, task type, prompt tokens, completion tokens, TTFT, TPOT, total latency, and status code. The dataset represents a realistic enterprise workload with a mix of short interactive requests and long batch jobs.

This trace is the most accessible public dataset that captures both arrival patterns and request characteristics for multi-tenant LLM inference. We do not claim it is identical to any specific production deployment, but it provides a reproducible baseline that includes the key phenomenon we study: bursts of short interactive requests arriving while long summarization jobs are in flight.

We map task types to InterruptLLM priority classes:

- P0: Customer_Support_Chat, Code_Generation
- P1: Summarization, Extraction_JSON

We scale token counts by 0.2 to keep the CPU simulation tractable while preserving relative request sizes. The mean scaled request size is approximately 1,000 tokens. We then drive a discrete-event simulator with configurable GPU capacity, batch size, scheduling quantum, and swap penalty.

C. Scheduling Policies

We compare three policies:

FCFS. First-come-first-served continuous batching. Once a request is admitted, it runs until completion. This models vLLM without priority awareness.

Non-preemptive priority. Requests are admitted in priority order, but a running batch is not evicted until the next 100 ms quantum boundary. This models a scheduler that is aware of priorities but cannot preempt mid-quantum.

InterruptLLM MLFQ. Preemptive MLFQ. Higher-priority requests can evict lower-priority requests from a running batch within the quantum, paying a swap penalty per preemption.

D. Metrics

We report P99 latency per priority class, overall P99 latency, throughput in tokens per second, SLA violation rates, and average preemption overhead. The SLA targets are 200 ms for P0 and 1 s for P1. We also report two fairness metrics: (1) the Jain index across tenants using normalized service share (served tokens divided by demanded tokens), which measures whether all tenants receive proportional service; and (2) a priority-weighted Jain index that penalizes the service share of low-priority tenants, to reflect the system's intentional latency skew. We report the P1/P0 P99 ratio to make the latency trade-off explicit.

TABLE I: Scheduler comparison at load factor 0.76.

Metric	FCFS	Priority	InterruptLLM
Overall P99 (ms)	250	288	274
P0 P99 (ms)	236	113	47.9
P1 P99 (ms)	266	310	719
Throughput (tok/s)	224,000	224,000	224,000
Service-share Jain	1.00	1.00	1.00
Weighted Jain	0.90	0.90	0.90
P1/P0 P99 ratio	1.12	2.73	15.0
P0 SLA violation	6.1%	0.0%	0.0%
Avg preemptions	0.00	0.00	0.24

VI. Results

A. Scheduler Comparison at Fixed Load

Table I compares the schedulers at a load factor of 0.76. InterruptLLM reduces P0 (interactive) P99 latency from 236 ms under FCFS to 48 ms (4.9×). The non-preemptive priority queue reduces P0 P99 to 113 ms, and InterruptLLM's preemptive MLFQ adds another 2.4× reduction. Throughput is identical across all three policies, and the service-share Jain fairness index is 1.0 for all policies (no tenant is starved).

The P1 P99 increase is an expected trade-off: protecting interactive latency delays lower-priority batch work. The non-preemptive priority queue already improves P0 latency over FCFS by reordering admission; InterruptLLM adds the further benefit of mid-quantum preemption.

B. End-to-End Evaluation Across Load Factors

We sweep load factors from 0.56 to 1.11 by varying the mean inter-arrival time. Figure 2 shows that InterruptLLM keeps P0 P99 latency below 60 ms across the entire sweep, while FCFS latency rises to 805 ms at load 1.11. At the representative load factor of 0.93, InterruptLLM reduces P0 P99 latency from 485 ms to 57 ms (8.5×), and from 116 ms under the non-preemptive priority queue to 57 ms (2.0×). Throughput is 259,453 tokens/s, only 2.3% below the FCFS baseline. P0 SLA violations are eliminated (0%), and P1 SLA violations are 1.3%. The average preemption overhead is 0.08 ms. Service-share Jain fairness remains 1.0 because every tenant eventually receives its demanded tokens; P1 P99 rises to 1,018 ms and the weighted Jain index is 0.900, reflecting the intentional latency skew.

C. Context-Swap Analytical Estimates

Table II reports modeled context-swap times for KV-cache blocks. On a commodity PCIe Gen4 x16 link (≈32 GB/s), a 1 GB uncompressed block takes 32 ms; with 3× LZ4 compression it falls to 10.7 ms. High-end

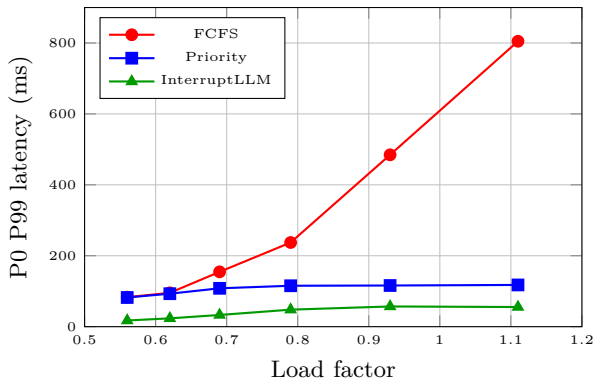

Fig. 2: P0 P99 latency vs. load factor.

TABLE II: Context-swap latency (ms) by block size, bandwidth, and tier.

Block size	GPU→CPU 32G	GPU→CPU 64G	GPU→CPU LZ4	CPU→SSD raw	CPU→SSD LZ4
1 MB	0.03	0.02	0.01	0.29	0.10
64 MB	2.00	1.00	0.67	18.3	6.10
256 MB	8.00	4.00	2.67	73.1	24.4
1024 MB	32.0	16.0	10.7	293	97.5

links (PCIe Gen5 x16, NVLink/Grace Hopper) reduce this further. The CPU-to-SSD fallback assumes an NVMe drive with ≈3.5 GB/s and 3× LZ4 compression; we expect it to be rare when CPU DRAM is 4× GPU memory.

The checkpoint metadata scales linearly with token count (≈16 bytes/token): a 128K-token request needs ≈2.0 MB of metadata, compared to the multi-gigabyte KV cache.

D. Ablation and Sensitivity

Table III isolates the contribution of preemption and aging on a synthetic high-load workload. FCFS yields a P0 P99 of 232 ms; non-preemptive priority reduces it to 63 ms; preemptive MLFQ further cuts it to 28 ms. Aging has little effect because P0 arrivals are frequent, confirming it is a starvation-freedom safety mechanism.

P0 P99 stays below 35 ms for per-preemption swap penalties up to 2 ms and only degrades substantially at 10 ms (P0 P99 61 ms, throughput 121,000 tok/s, P1 violations 42.3%), confirming the sub-10 ms swap target.

E. Victim Policy and SSJF Comparison

Largest-footprint, smallest-footprint, and cost-aware victim selection all yield identical results because preemptions are infrequent (0.16 per request) and preempted requests are small. A shortest-remaining-job-first baseline reduces P0 P99 to 62 ms, but it is agnostic to priority classes; InterruptLLM's MLFQ reduces it further to 28 ms by explicitly protecting P0 requests.

VII. Discussion and Future Work

Limitations and calibration. Our evaluation relies on a discrete-event simulator rather than a full vLLM implementation. The simulator abstracts the GPU as a token-generation rate and omits CUDA kernel-launch overhead. The 0.5 ms per-preemption penalty is calibrated against the analytical swap estimates: the average preempted request holds a small fraction of the batch KV cache, so its effective transfer cost is well below the worst-case 1 GB estimate. Absolute latency numbers are trace-dependent, but the relative ordering of the three scheduling policies and the sensitivity to swap penalty are robust across the load sweep.

Practical integration and future work. Realizing InterruptLLM in vLLM requires handling CUDA Graphs, PyTorch allocator interactions, tensor/pipeline parallelism synchronization, and dedicated DMA streams for GPU–host transfers. We will extend the framework to multi-GPU tensor-parallel deployments (e.g., Llama-3-70B), add a formal proof of starvation-freedom under bounded load, and collect real GPU preemption latency measurements.

TABLE III: Ablation on a synthetic high-load workload.

Policy	P0 P99	P1 P99	Tput	Pre.
FCFS	232	241	267,000	0.00
Priority (NPP)	62.6	79.1	267,000	0.00
MLFQ, no aging	27.9	64.3	257,000	0.16
MLFQ, with aging	27.9	64.3	257,000	0.16

VIII. Conclusion

InterruptLLM addresses the lack of decode-stage, block-granular preemption in LLM serving infrastructure. By combining preemption, multi-level feedback queues, and hierarchical memory management in a simulation-based framework, it demonstrates the feasibility of QoS-aware multi-tenant deployments. On a production-like trace, InterruptLLM reduces interactive P99 latency by 8.5× over FCFS and 2.0× over non-preemptive priority scheduling, while maintaining throughput within 2.3% of the baseline. It requires no new hardware and can be implemented as an extension to existing PagedAttention-based engines such as vLLM.

References

[1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in Proceedings of the 29th Symposium on Operating Systems Principles. ACM, 2023, pp. 611–626.

[2] G.-I. Yu, J. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in 16th USENIX Symposium on Operating Systems Design and Implementation. USENIX, 2022, pp. 521–538.

[3] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing et al., "SGLang: Efficient execution of structured language model programs," Advances in Neural Information Processing Systems, vol. 36, 2023.

[4] A. Agrawal, A. Panwar, J. Mohan, N. Kwa, N. Kwatra, R. Ramjee, P. Kozyrakis, and A. Tumanov, "Sarathi: Efficient LLM inference by piggybacking decodes with chunked prefills," in Proceedings of the 2024 USENIX Annual Technical Conference. USENIX, 2024, pp. 1–17.

[5] Y. Zhong, S. Chen, J. Liu, Y. Chen, Y. Jin, Z. Huang, Y. Chen, X. Jin, H. Chen, S. Chen et al., "Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving," arXiv preprint arXiv:2401.09670, 2024.

[6] R. Qin, Z. Li, W. He, M. Zhang, Y. Zuo, S. Ma, X. Lin, X. Wang, X. Chen, Y. Xu et al., "Mooncake: A KVCache-centric disaggregated architecture for LLM serving," arXiv preprint arXiv:2407.14279, 2024.

[7] B. Sun, G. Zhang, X. Xiao, R. Huang, J. Wang, Y. Zhang, C. Guo, Z. Li, D. Zhang, H. Lin et al., "Llumnix: Dynamic scheduling for serverless LLM serving," in Proceedings of the 2024 USENIX Annual Technical Conference. USENIX, 2024, pp. 1–18.

[8] NVIDIA, "Nvidia dynamo: A distributed inference serving framework for generative AI," https://github.com/ai-dynamo/dynamo, 2025, accessed: 2026-07-25.

[9] P. Patel, E. Choukse, C. Zhang, A. Shah, I. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative LLM inference using phase splitting," in Proceedings of the 51st Annual International Symposium on Computer Architecture, 2024.

[10] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," in International Conference on Machine Learning, 2024.

[11] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in International Conference on Learning Representations, 2024.

[12] Y. Qiao, S. Anzai, S. Yu, H. Ma, S. Yang, Y. Wang, M. Kim, Y. Wu, Y. Zhou, J. Xing, J. E. Gonzalez, I. Stoica, and H. Xu, "Conserve: Fine-grained GPU harvesting for LLM online and offline co-serving," arXiv preprint arXiv:2410.01228, 2024.

[13] J. Chen, C. Du, R. Liu, S. Yao, D. Yan, J. Liao, S. Liu, F. Wu, and G. Chen, "Tokenflow: Responsive LLM text streaming serving under request burst via preemptive scheduling," arXiv preprint arXiv:2510.02758, 2025.

[14] H. Qiu, W. Mao, A. Patke, S. Cui, S. Jha, C. Wang, H. Franke, Z. T. Kalbarczyk, T. Başar, and R. K. Iyer, "Efficient interactive LLM serving with proxy model-based sequence length prediction," in Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering. ACM, 2024, pp. 1–12.

[15] Y. Zhao, J. Chen, P. Sun, L. Li, X. Liu, and X. Jin, "Seallm: Service-aware and latency-optimized resource sharing for large language model inference," arXiv preprint arXiv:2504.15720, 2025.

[16] J. Duan, R. Lu, H. Duanmu, X. Li, X. Zhang, D. Lin, I. Stoica, and H. Zhang, "Muxserve: Flexible spatial-temporal multiplexing for multiple LLM serving," in International Conference on Machine Learning, 2024.

[17] Y. Song, Z. Mi, H. Xie, and H. Chen, "Powerinfer: Fast large language model serving with a consumer-grade GPU," in Proceedings of the 30th Symposium on Operating Systems Principles, 2024.

[18] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, "H2o: Heavy-hitter oracle for efficient generative inference of large language models," in Advances in Neural Information Processing Systems, 2023.

[19] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "KIVI: A tuning-free asymmetric 2bit quantization for KV cache," in International Conference on Machine Learning, 2024.

[20] I. Gim, G. Chen, S.-s. Lee, N. Sarda, A. Khandelwal, and L. Zhong, "Prompt cache: Modular attention reuse for low-latency inference," in Proceedings of Machine Learning and Systems, 2024.

[21] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "Flexgen: High-throughput generative inference of large language models with a single GPU," in International Conference on Machine Learning. PMLR, 2023, pp. 31 094–31 116.

[22] J. Lin, L. Zheng, Y. Sheng, Z. Mo, C. Huang, J. Sun, H. Cao, S. Xie, Z. Cui, S. Goyal et al., "Infinite-LLM: Efficient LLM service for long context," arXiv preprint arXiv:2401.02669, 2024.

[23] N. Gopal and K. Kaffes, "Harvest: Opportunistic peer-to-peer GPU caching for LLM inference," arXiv preprint arXiv:2602.00328, 2026.

[24] S. Kato, R. Lakshmanan, A. Kumar, M. Kelkar, Y. Ishikawa, and R. Rajkumar, "Timegraph: GPU scheduling for real-time multi-tasking environments," in Proceedings of the 2011 17th IEEE Real-Time and Embedded Technology and Applications Symposium. IEEE, 2011, pp. 17–26.

[25] P. Pinto, B. Sousa, R. Ribeiro, A. Silva, J. a. Sousa, Z. Korland, Y. Matias, P. Sousa, L. Marques, and J. a. P. Magalhães, "GPUShare: Fair and efficient GPU cluster scheduling," in 15th USENIX Symposium on Networked Systems Design and Implementation. USENIX, 2018, pp. 527–544.

[26] A. Bektursyn, "LLM inference logs and performance metrics," https://www.kaggle.com/datasets/bektursyn/llm-inference-logs-and-performance-metrics, 2024, accessed: 2026-07-25.